Western Norway University of Applied Sciences

Beyond Vibe Coding

From Concept to MVP

Spec-driven, AI-assisted Software Engineering



Asst. Prof. Per Christian Engdal
Western Norway University of Applied Sciences
2026

The Next Level of Abstraction In S.D.

HIGHER ABSTRACTION

2020s Level 5	**Agentic Development** Natural language, Markdown specs, Gherkin (BDD)	Intent, constraints & verification	Specifications → Agents → Verified system
2000s Level 4	**Model-Driven Development (MDD)** UML, SysML, OCL, DSLs	Models & transformations	UML / DSL → Code generation
1980s Level 3	**Object-oriented development** Smalltalk, C++, Java, C#	Domain concepts	order.calculateTotal()
1960s Level 2	**High-level languages** Fortran, COBOL, Lisp, ALGOL, C	Algorithms	total = price * quantity
1950s Level 1	**Machine code / Assembly** Machine code, Autocoder, IBM 704 assembly	Instructions	MOV AX, 5



Kent Beck

KENT BECK

“I’m not a great programmer; I’m just a good programmer with great habits.”

WHO HE IS

- Created Extreme Programming (XP) and test-driven development (TDD)
- Co-created the JUnit testing framework and pioneered software design patterns
- One of the 17 original signatories of the Agile Manifesto (2001)

Agenda

01	**From Vibe Coding To Spec-Driven Development** Why a plausible app is not a finished app, and why specs address it	
02	**Agentic Development: Two loops** The product loop and the execution loop	
03	**Agentic Development: Harness and engineering control** Rules, permissions and checks in the repo	
04	**Agentic Development: The product loop** From requirements and evidence, to ready specs and user stories	**Demo 1 · product framing**
05	**Agentic Development: The execution loop** Building the MVP - from a ready story to an accepted increment	**Demo 2 · plan, build and verify**
06	**Agentic Development: Where we are heading** Formal specs, stronger checks and risk-based review	
07	**Agentic Development: The MVP playbook** Seven steps and the harness you take home, with Lovable as an option	

Take It Home: Everything Is On GitHub

WHAT YOU GET AFTER THE LECTURE

Video lecture
The full lecture, recorded, to watch again at your own pace

Demos
Product framing, and plan, build and verify, step by step

Two harnesses
Rules, permissions and checks for both tracks, ready to copy

pcehvl.github.io/MakersForge-2026

No need to take notes. **Follow the ideas now, use the material later.**

From Vibe Coding To Spec-Driven Development

Agentic Development

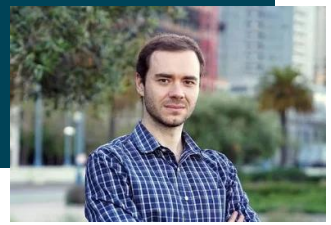
01

What Vibe Coding Means

THE ORIGINAL MEANING

“Fully give in to the vibes, embrace exponentials, and forget that the code even exists.”

Andrej Karpathy, February 2025



IN PRACTICE

Describe it
Tell the model in plain language what you want

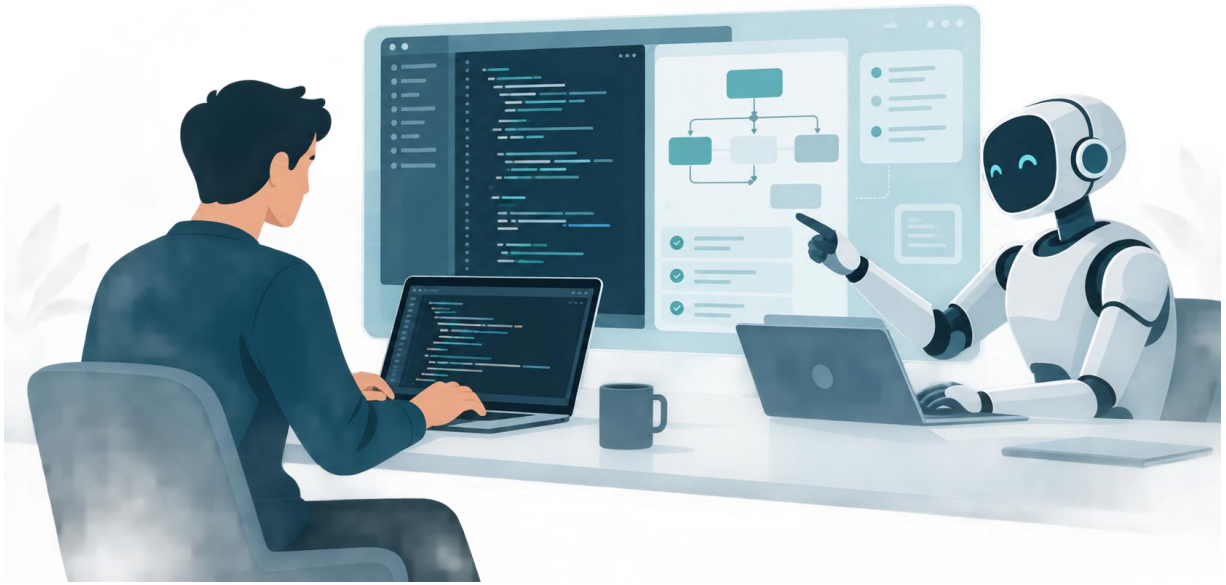
Try it
Judge the result by running it, not by reading it

Ask for changes
Repeat until it looks right

Vibe coding means judging the result on screen. Nobody reads the code.

Source: Andrej Karpathy, post on X, 2 February 2025

With An Agent, You Get A New Coworker



WHAT YOUR NEW COWORKER BRINGS

Knows more
Broad knowledge across every domain

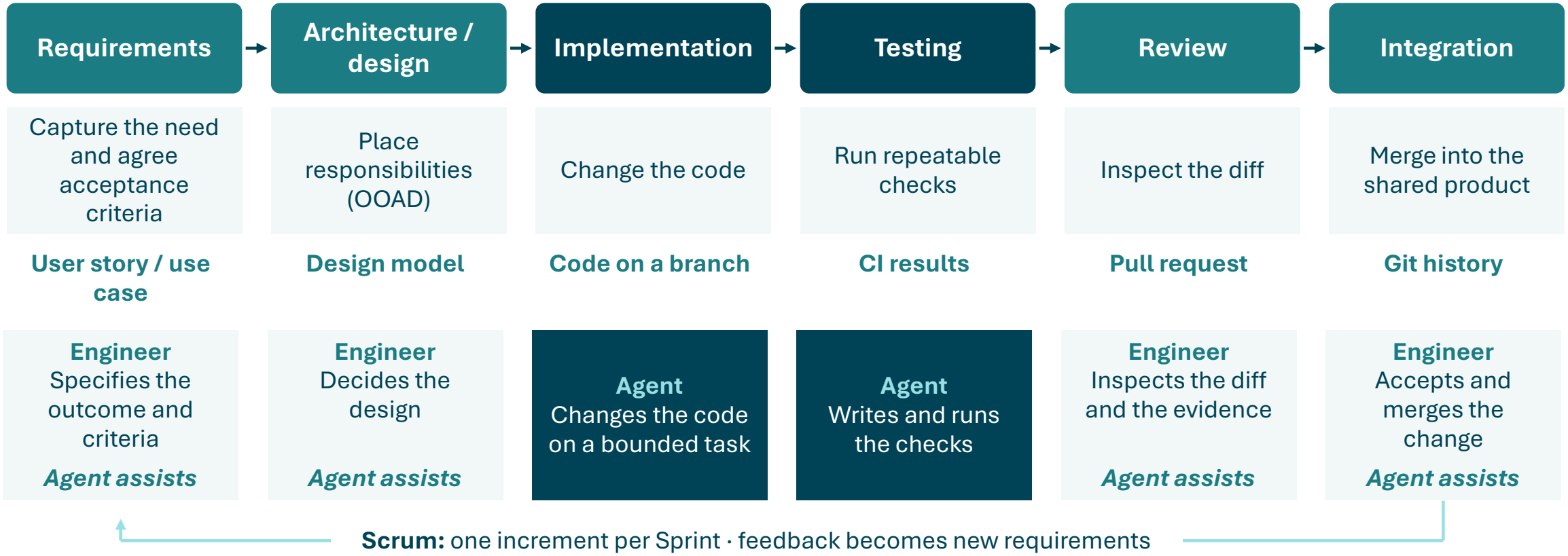
Has seen more
More code and cases than one person sees

Has the tools
Searches, writes, runs and tests in minutes

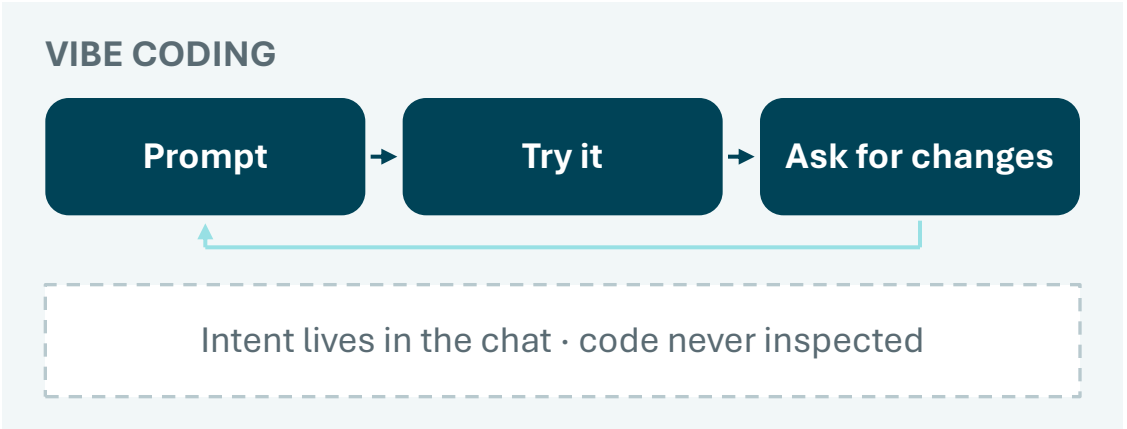
Gets better
High quality today, and each model is better

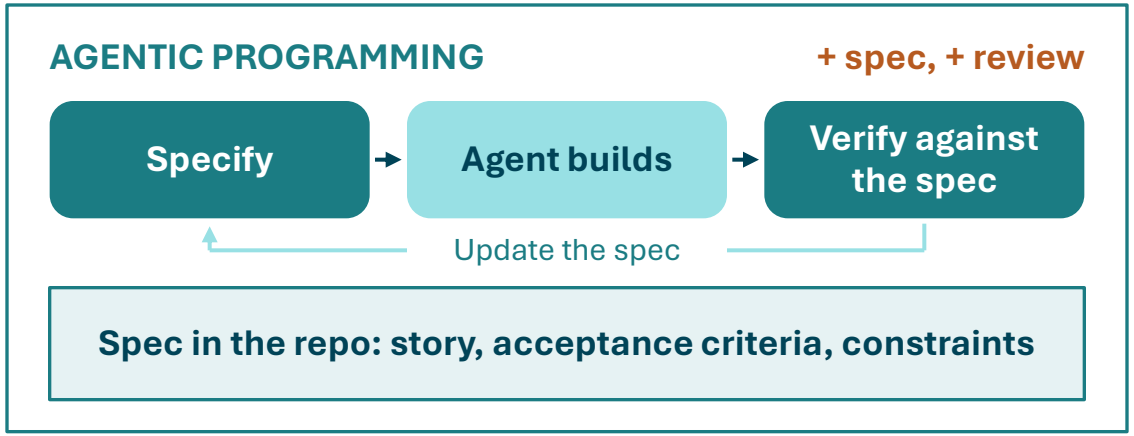
The question is no longer what the agent can do. It is what you ask it to do.

The Software Engineering Workflow - with Agents



Vibe Coding And Agentic Programming





Anyone can do it, even without programming

Each session starts from scratch. Often weak on maintainability, correctness and security.

Needs engineering knowledge

The spec carries intent across sessions and agents. The engineer verifies structure, correctness and security.

When is vibe coding fine?
When all three hold:

- Throwaway
- Low stakes
- Errors show on use

Agents Write The Code. We Write The Spec

THE HARD PART

“The hardest single part of building a software system is deciding precisely what to build.”

Fred Brooks, No Silver Bullet, 1987



WHAT THE SPEC GIVES US WITH AGENTS

Same prompt, different code
Stable target: code varies, the criteria do not

The agent acts on its own
Control: scope and constraints set the limits

No criteria, no done
Verifiable: the criteria give PASS or FAIL

Faster than you can read
Review: check against the spec, not every line

Agents handle the coding. **The spec is the hard part, and it is ours.**

Source: Frederick P. Brooks, “No Silver Bullet: Essence and Accidents of Software Engineering”, 1987

What We Mean By A Spec

WHAT A SPEC CONTAINS

User story
Who needs what, and why

Acceptance criteria
Testable conditions: PASS or FAIL

Constraints
Security, privacy and compliance rules to meet

Out of scope
What the agent must not change

EXAMPLE **specs/login-lockout.md**

Story
As a user, I want my account locked after five failed logins, so attackers cannot guess my password.

Acceptance criteria
Five failed attempts lock the account for 15 minutes
A successful login resets the counter

Constraints Every lockout is logged for audit

Out of scope Password reset flow

A spec feeds every prompt, but it lives in the repo and outlives the chat.

Architecture and tech choices go in **plan.md**

Two Files: What To Build, Then How

spec.md What and why

CONTAINS
Story, acceptance criteria, constraints, scope

REVIEWED BY
Product owner and engineer

CHANGES WHEN
The need changes

THE AGENT USES IT TO
Know when the work is done

→

plan.md How

CONTAINS
Architecture, tech stack, data model, interfaces

REVIEWED BY
Engineer and tech lead

CHANGES WHEN
The design or technology changes

THE AGENT USES IT TO
Know how to build it

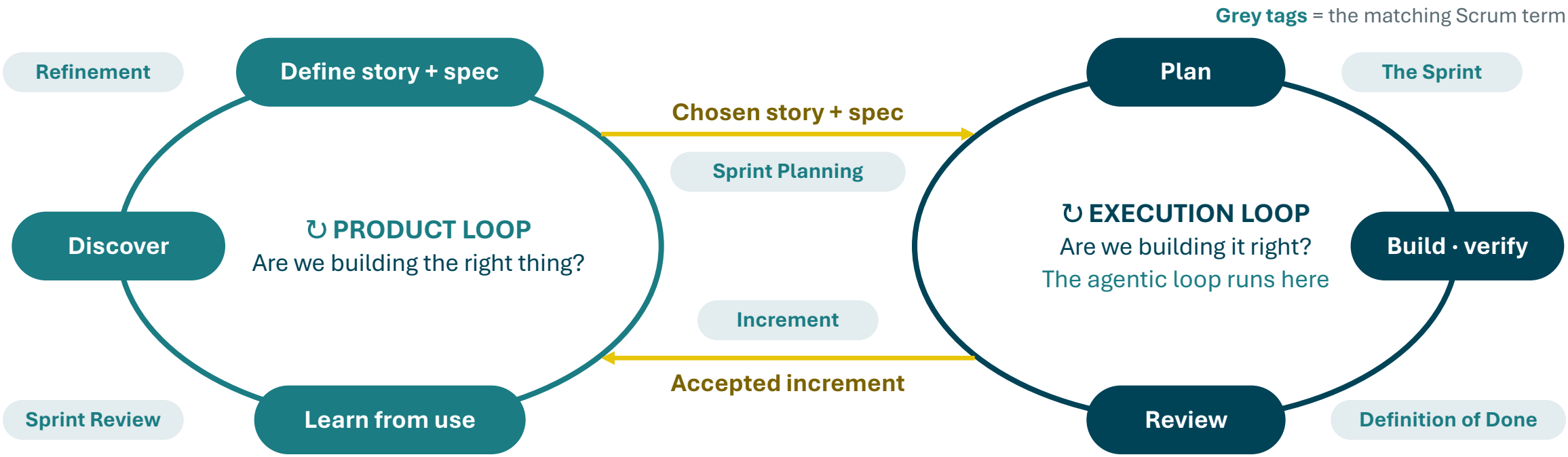
One reason to change per file. Swap the tech without rewriting the need.

Agentic Development: - Two Loops

Agentic Development

02

The Product Loop And The Execution Loop



Led by people: users and the Product Owner
They decide what to build and accept the value

Led by the System Engineer: Planner and Coder do the work
Approves the plan, owns the gate, merges

Agents speed up the execution loop. Only people using the product can close the product loop.

The Product Owner And The System Engineer

PRODUCT OWNER · PRODUCT LOOP

Owns the what and the why

- Vision, scope and priority
- Stories and acceptance criteria
- Answers refinement questions
- Accepts the increment in the Preview
- Decides on feedback

Ready story →

← Accepted increment

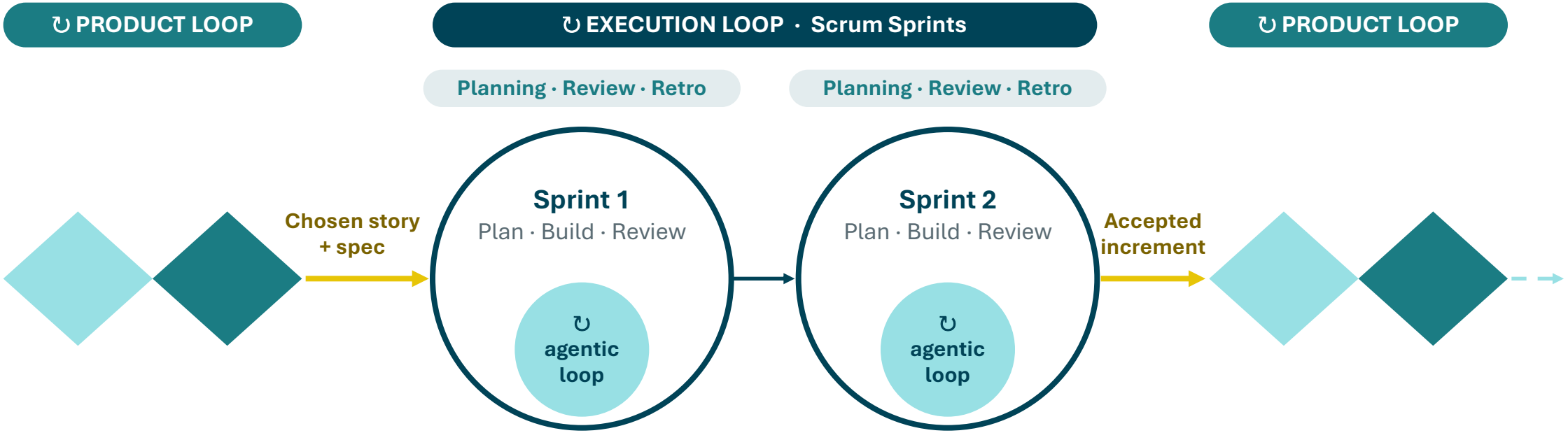
SYSTEM ENGINEER · EXECUTION LOOP

Owns the how and the quality

- Approves the Planner's plan
- Steers the Coder, stops drift
- Owns the gate and Definition of Done
- Reviews the code in the pull request
- Merges and deploys after acceptance

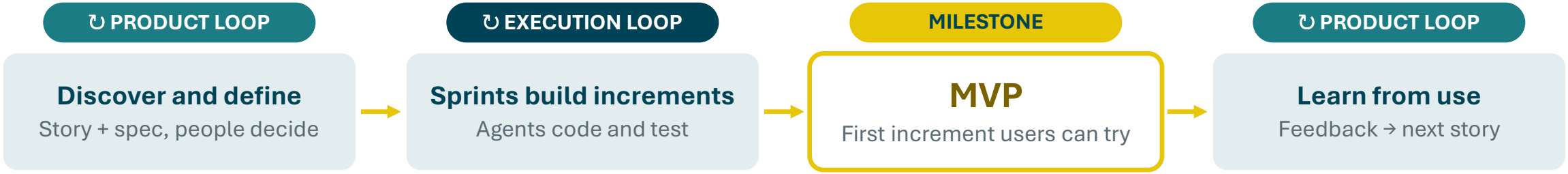
Two roles, as in Scrum. In this course, you play both: decide as PO, build and merge as System Engineer.

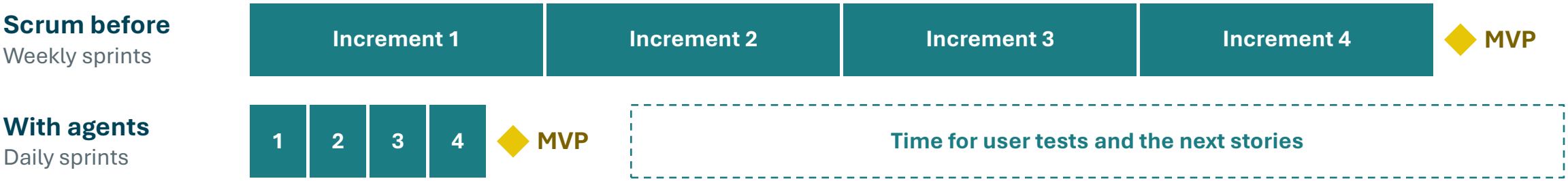
Design Thinking Finds What - Scrum Builds It



Design Thinking decides what to build. Scrum builds it in increments. Agents work inside each Sprint.

With Agents, We Can Reach The MVP Sooner





The MVP closes the first execution loop. Agents give us the capacity to get there faster.

Agentic Development - Harness And Engineering Control

Agentic Development

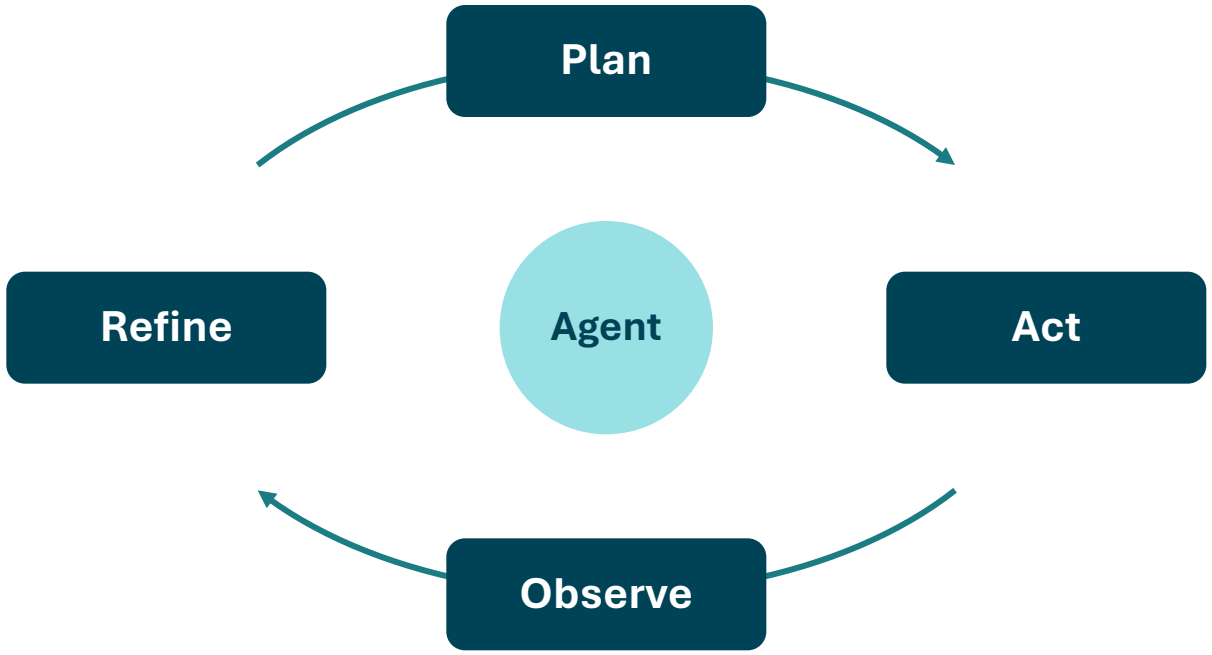
03

How An Agent Works

WHAT GOES IN

Prompt
The task, stated in words

Context
Code, specs and earlier decisions



WHAT IT CAN USE

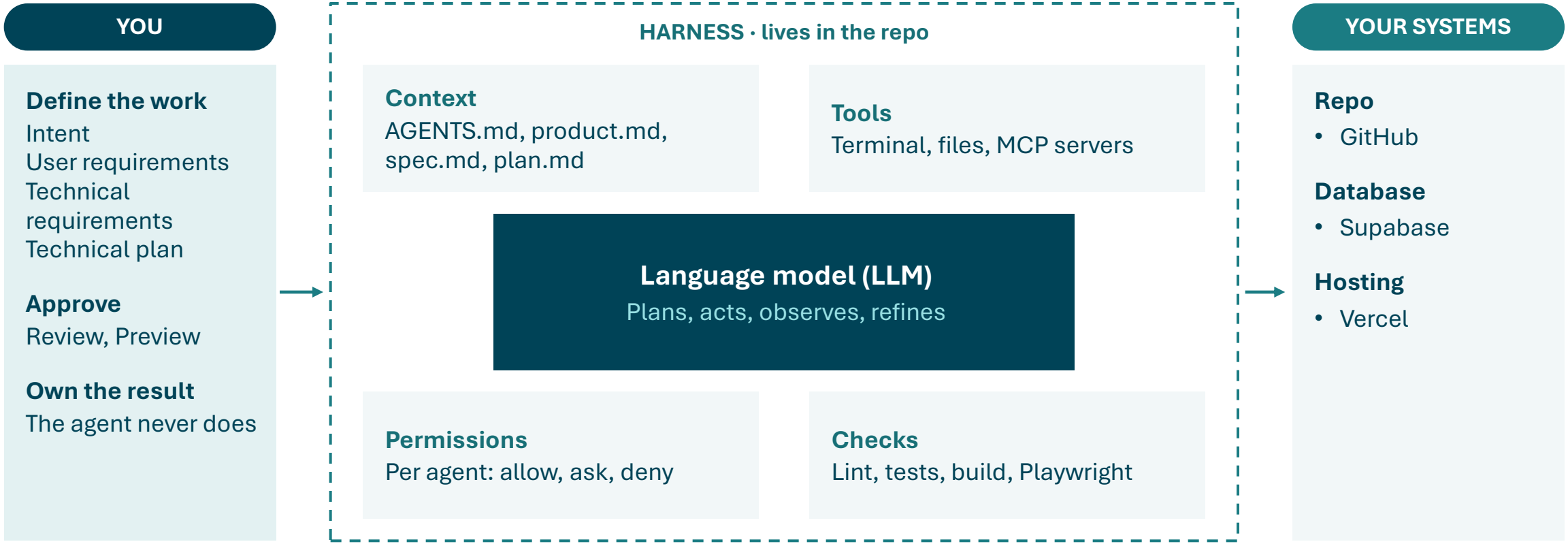
Tools
Run tests, edit files, call APIs (e.g. MCP)

Memory
Notes carried between steps

The engineer stays outside the loop: sets the goal, grants the tools, approves the result.

Human-in-the-loop: the agent stops for review before a change is accepted.

Agent = LLM + Harness



Same model, better harness, better result. The harness is the part you engineer.

The Harness Lives In The Repo

```
studymatch/
├─ AGENTS.md
├─ product/
│  ├─ product.md
│  └─ stories/US-001-…/spec.md plan.md
├─ docs/design-thinking/
├─.opencode/
│  ├─ agents/  3 agents
│  └─ skills/  6 skills
├─ src/ · e2e/ · supabase/
└─ .github/  PR template
```

CONTEXT	**AGENTS.md · product/ · docs/** What the agent knows before it starts
ROLES	**.opencode/agents/** Product Assistant: what · Planner: how · Coder: build
SKILLS	**.opencode/skills/** Six methods, e.g. product-management
PERMISSIONS	**.opencode/agents/*.md** Front-matter: what each agent may edit and run
CHECKS	**e2e/ · npm scripts** Lint, types, tests, build, Playwright before every PR

If it is not in the repo, the agent does not know it. Every rule is a file you can review and version.

3 Agents In Our Template-Harness ：
- Separate Roles, Skills And Files

AGENT · PRODUCT ASSISTANT

Why and what

- Turns discovery into product framing
- Writes and refines user stories with ACs
- Sets story status: backlog, ready, done
- Classifies feedback for the PO

Skills: product-management, feedback-management

Writes: product.md and stories/<id>/spec.md

Never: code or technical plans

AGENT · PLANNER

How

- Plans exactly one Ready story
- Checks the code before planning
- Maps every AC to a test
- Names risks and human checkpoints

Skills: plan-story

Writes: stories/<id>/plan.md

Never: implements or changes the ACs

AGENT · CODER

Build and verify

- Opens the Issue and a feature branch
- Implements the plan with tests
- Runs the full gate, reports each AC
- Opens the PR, deploys on acceptance

Skills: implement-story, ui-design, deploy

Writes: code, tests and the pull request

Never: edits requirements or merges alone

The Product Owner owns the what and accepts the value. The System Engineer owns the how, the quality gate and the merge.

An Agent Is A Role, A Skill Is A Method

Agent file Who and what

WHERE
.opencode/agents/product-assistant.md

CONTAINS
Role, limits, permissions, hand-off

LOADED
Always, while the agent runs

ANSWERS
Who acts, and what it may touch

Skill How

WHERE
.opencode/skills/product-management/

CONTAINS
Description, steps, template

LOADED
Only when the task needs it

ANSWERS
How the task is done, every time

Keep roles small and methods reusable. One agent can use several skills.

Anatomy Of An Agent File

FILE `.opencode/agents/product-assistant.md`

FRONT-MATTER enforced by the harness

```
---
description: Product Assistant. Why and what.
permission:
  edit: { "product/**": allow, "*": deny }
  bash: deny
---
```

PROMPT guidance for the model

```
You own the what and the why, never the how.
Use the product-management skill for stories.
Write: product.md and stories/<id>/spec.md
Never: code or technical plans.
```

1 Permissions

May edit product/ only. No shell.

2 Role and limits

What and why. Never code or plans.

3 Hand-off

Delivers spec.md for the Planner.

The front-matter is enforced by the harness. The prompt sets the role.

Simplified excerpt. The full file is in the template repo.

Guardrails: What The Agent Cannot Do

ENFORCED

The harness blocks it

Deterministic. The agent cannot do it, whatever the prompt says.

- Permissions per agent: allow, ask, deny
- Branch protection: no push to main
- Secrets only in .env, never in code or prompts
- No migrations or deletes against production

CHECKED

The gate catches it

Automatic. The change fails before it reaches a human.

- Lint, types, tests, build, Playwright
- Row Level Security tested per table
- Secret scanning and dependency audit
- Out-of-scope files flagged in the diff

APPROVED

A human decides it

The agent stops and waits.

- Engineer approves plan.md
- Engineer reviews the pull request
- PO accepts in the Preview
- Engineer merges and deploys

A rule in a prompt is a wish. A rule in the harness is a guardrail.

Anatomy Of A Skill

FILE .opencode/skills/product-management/SKILL.md

```
---
name: product-management
description: Write user stories with ACs.
  Use when a story is created or made ready.
---
1. Restate the user need in one sentence
2. Write the story and testable ACs
3. Add constraints and out of scope
4. Ask the PO about anything unclear
Template: spec.md
```

1 Description

Decides when the agent loads it

2 Steps

The same method, every time

3 Template

Gives spec.md its four parts

Six skills, three agents. Fix a skill once, and every agent gains.

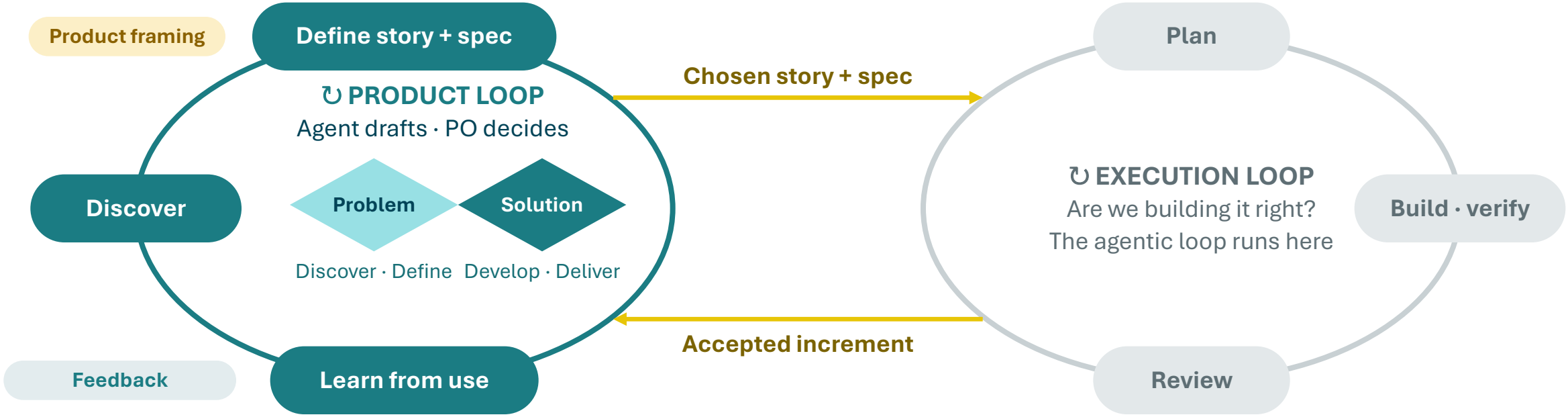
Simplified excerpt. The full file is in the template repo.

Agentic Development: -The Product Loop

Agentic Development

04

Product Loop: Product Framing

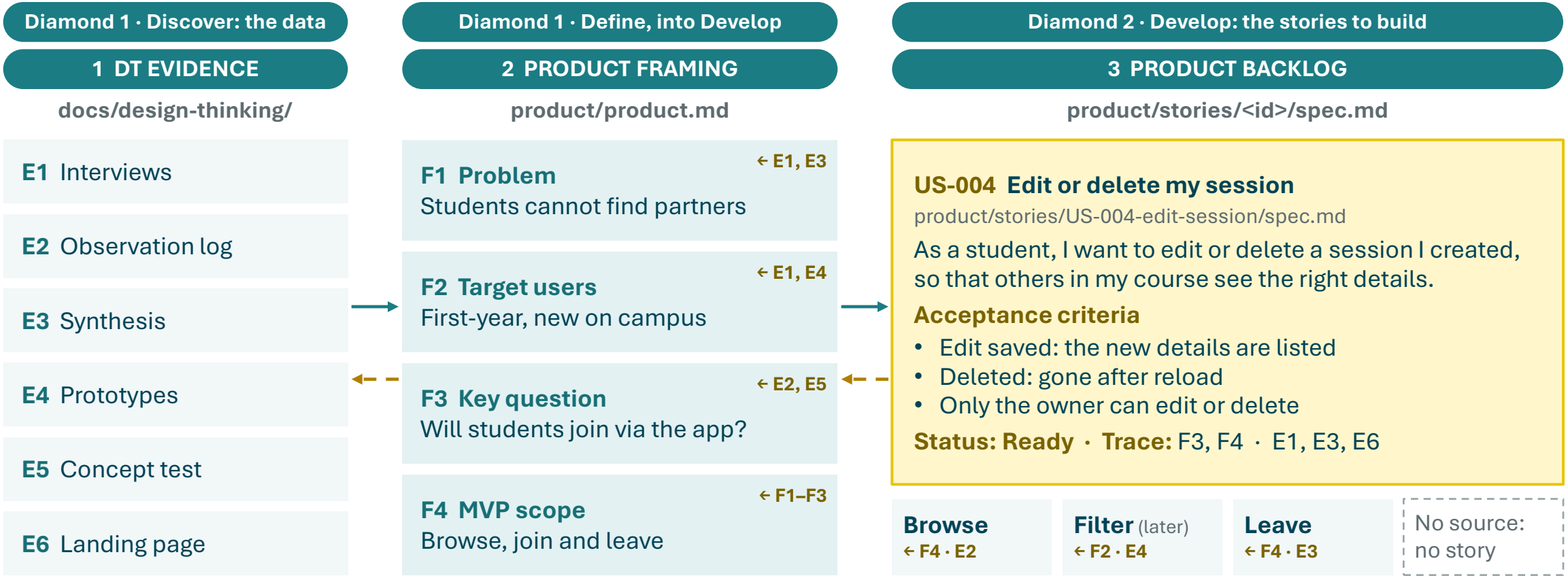


Artefacts: product.md · stories/<id>/spec.md
People decide value; the Product Assistant drafts and asks

Execution loop: next section
Planner and Coder build one Ready story

The product loop decides what to build. Agents help at every step, but the PO decides.

Product Loop: Product Framing



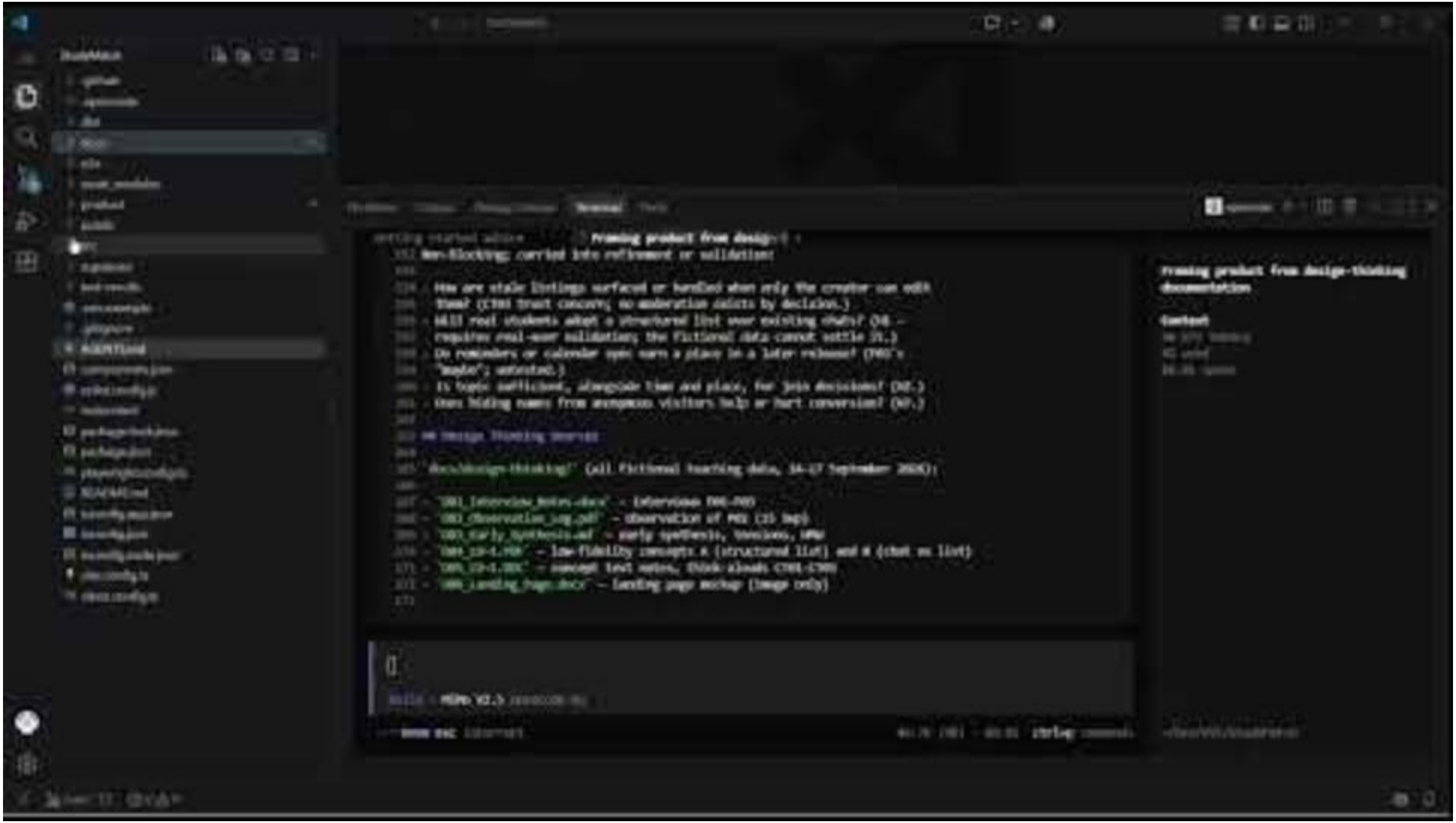
One story, one spec.md. It holds the status and traces back to the evidence.

Demo 1

From Discovery To Product Framing

Open: docs/design-thinking/

D1



Product Framing Gave Us The Product Backlog

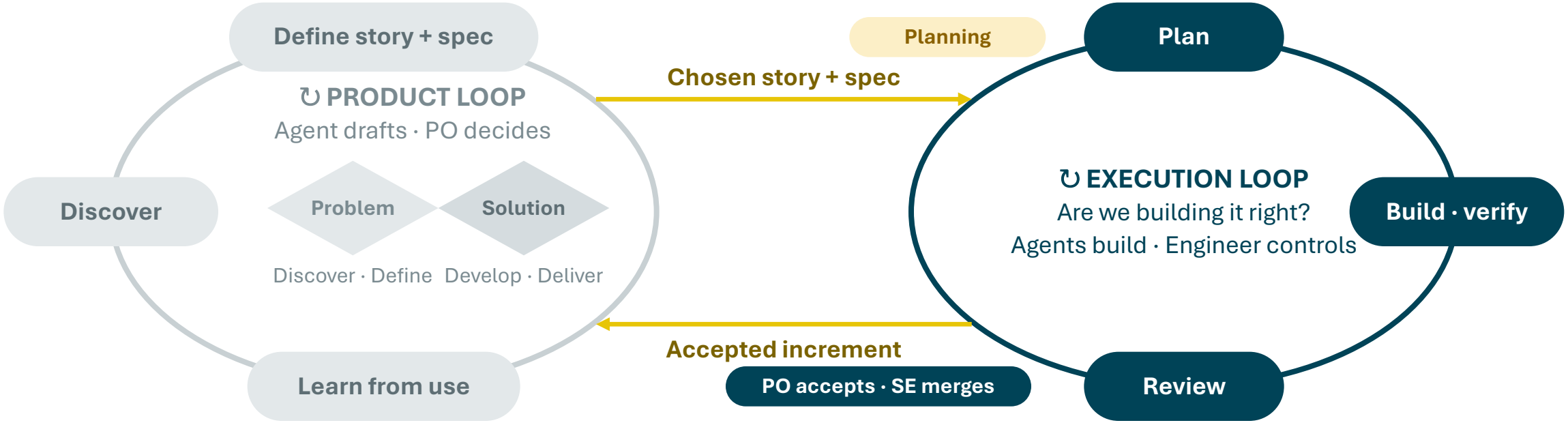
← From product framing	Scrum terms, and where they live in the repo	
IN SCRUM	**IN OUR REPO**	**WHERE**
Product Backlog	product/stories/, one folder per user story	Framing
Product Backlog Item	One spec.md per user story, status: backlog	Framing
Backlog refinement	The Product Assistant asks, the PO answers	Refinement
Definition of Ready	Five Ready checks pass, then status: ready	Refinement

The Product Backlog exists. Next, refinement makes one item Ready.

Agentic Development: The Execution Loop

05

Execution Loop: From Ready Story To Increment



Product loop: done for this story
One Ready story, specified in spec.md

Artefacts: plan.md · branch · PR
The Planner plans, the Coder builds and verifies

The execution loop builds it right. Agents do the work, the System Engineer controls it, the PO accepts it.

Agents Build, Engineer Controls, PO Accepts

LOOP	Execution loop · The Sprint: plan, build and verify one story				→ Product loop
	1 Plan	2 Implement	3 Test	4 Pull request	5 Accept
AGENTS Planner, Coder	Planner reads spec.md and the code, writes the plan	Coder opens a GitHub Issue and a branch, then builds	Coder runs the full gate and checks every AC	Coder self-reviews, opens the PR and stops	On the engineer's go, the Coder deploys
HUMAN Engineer · PO	**Engineer: check the plan fits the story**	Engineer: watch, stop it if it drifts	Engineer: read the AC verdicts	**Engineer reviews the code, PO tries the Preview**	**PO accepts, engineer merges**
OUTPUT	plan.md	Issue + branch	AC: PASS / FAIL	Pull request	Deployed MVP

One Ready story at a time. The Coder stops at the PR; the PO accepts and the System Engineer merges.

The Planner Agent Turns A Story Into A Plan

1 spec.md · why and what

US-004 Edit or delete my session

As a student, I want to edit or delete a session I created, so that others in my course see the right details.

Acceptance criteria

AC1 Edit saved and listed
AC2 Delete gone after reload
AC3 Only the owner can change it

2 plan.md · how

Planner writes only plan.md

- Existing system: facts, not guesses
- Approach and rejected options
- Migration, or the reason for none
- Every AC mapped to a test
- Human checkpoints it cannot fake

3 Implementation sequence

1 Migration and RLS policy
2 Edit form and save
3 Delete, gone after reload
4 Hide actions for others
5 Tests and Playwright journey

The plan holds the how, never the what. It may not weaken, redefine or expand an AC; a gap goes back to the PO.

Demo: Plan And Build
- From Story To Coded Solution

Execution loop · The Sprint: plan, build and verify one story

1 Plan	2 Branch	3 Build	4 Gate
The Planner writes plan.md for US-004. **Watch:** The engineer reviews: facts checked, every AC mapped.	The Coder opens a GitHub feature branch. **Watch:** The branch name carries the story ID.	The Coder implements the plan and writes the tests. **Watch:** Human prerequisites stay with the engineer.	Lint, types, tests, build and Playwright run locally. **Watch:** PASS or FAIL per AC.
Planner	Coder	Coder	Coder

One story, end to end. The agent builds; the plan and the quality gate keep it on the story.

Demo 2

Plan, build and verify US-004

D2

Agentic Development: Where We Are Heading

Future outlook

06

Effort Moves From Code To Spec And Checks

WHERE WE ARE HEADING

1 SPECIFY

Formal specs, not prompts

- E.G. BDD/Gherkin scenarios are the acceptance criteria
- The same scenarios run as automated tests
- The spec lives in the repo and is reviewed like code

2 VERIFY

Checks and guardrails

- Tests, static analysis and security scans on every change
- CI gates: no merge without green checks
- Sandboxes and permissions limit what the agent can touch

3 REVIEW

Review by risk, not by line

- **Read in full:** security, data, payments, public interfaces
- **Skim:** code covered by passing scenarios
- Keep changes small so the diff stays readable

We still read code, just not all of it. The spec and the checks tell us where to look.

Agentic Development: -The MVP Playbook

07

The MVP Playbook: Seven Steps, Two Loops

PRODUCT LOOP · The agent drafts, you decide as Product Owner

1 Collect
Gather design-thinking evidence about users and the problem

2 Frame
Who it is for, which problem, which outcome; you decide as PO

3 Story
One spec per story, with testable acceptance criteria (ACn)

4 Ready
Refine one question at a time until the story is Ready

EXECUTION LOOP · The agent plans and builds, you control as System Engineer

5 Plan
The agent proposes a technical plan; you approve it as engineer

6 Build and verify
The agent builds the smallest slice and checks every ACn

7 Accept and learn
Accept PASS or FAIL per ACn as PO, release, test with users

Feedback → new or revised stories. Same seven steps in both tracks. Only the tools in the execution loop differ.

Your Take-Home Harness: OpenCode Track

```
your-mvp/
├─ AGENTS.md
├─ product/
│  ├─ product.md
│  └─ stories/US-XXX-<slug>/
│     spec.md · plan.md · verification.md
├─ docs/
│  ├─ design-thinking/
│  └─ architecture.md · ui-guidelines.md
├─ .opencode/  agents/ · skills/
├─ src/ · e2e/
├─ supabase/migrations/
└─ .github/  PR template
```

product/ PRODUCT LOOP
product.md holds the framing. One folder per story is the backlog: no backlog.md, no Issues.

.opencode/ AGENTS AND SKILLS
Product Assistant, Planner and Coder, with six skills.

stories/<id>/plan.md · verification.md EXECUTION LOOP
The Planner's plan you approve as engineer, and the Coder's evidence you read before you accept as PO.

src/ · e2e/ · supabase/ · .github/ BUILD AND CHECKS
npm run verify: lint, types, tests, build and E2E, before the PR and again before merge.

One repo, both loops in OpenCode. The app code and its checks live next to the specs. Template: MF-MyMVP-Template.

Your Take-Home Harness: Lovable Track

```
your-mvp/
├─ AGENTS.md · opencode.json
├─ product/
│  ├─ product.md
│  └─ stories/US-XXX-<slug>/
│     spec.md · acceptance.md
├─ docs/
│  └─ design-thinking/
├─ .opencode/  agents/ · skills/
└─ lovable/
   ├─ skills/
   └─ workspace-knowledge.md

App code lives in the Lovable project
```

product/ PRODUCT LOOP · SHARED

product.md and one folder per story: spec.md goes out, acceptance.md comes back. Plan and verification stay in Lovable.

.opencode/ PRODUCT LOOP IN OPENCODE

Product Assistant drafts the specs. /handover sends spec.md to Lovable, /return brings acceptance.md back.

lovable/skills/ EXECUTION LOOP IN LOVABLE

/harness-init, /build-story, /accept-story and /publish, imported once per workspace.

lovable/workspace-knowledge.md RULES FOR LOVABLE

Pasted into workspace knowledge, so Lovable follows the same rules. It builds and checks each ACn on the preview.

A product repo in OpenCode, the app in Lovable. Template: MF-MyMVP-Lovable-Template (pre-release v0.1.0).

Take it home !

Scan the QR code for all documentation, a video lecture, demos and harnesses for both tracks.

Per Christian Engdal
Asst. Prof., Western Norway University of Applied Sciences
Email: per.christian.engdal@hvl.no
Phone: +47 98 26 93 01
LinkedIn: linkedin.com/in/pcengdal

https://pcehvl.github.io/MakersForge-2026

L.G.T.M: https://www.youtube.com/watch?v=3TNpOD6bov8